🌐 Module 06: API and Java Web Token Penetration Testing

```
# Exercise 04: Exploiting NoSQL Injection for Denial-of-Service
```

Scenario

NoSQL databases (such as MongoDB) process API queries dynamically by passing user-supplied input directly into backend query expressions or JavaScript evaluation contexts (e.g., `$where` or map-reduce functions). When API endpoints fail to sanitize or type-check input parameters, attackers can inject JavaScript-based NoSQL statements.

Injecting execution delay functions like `sleep(integer_ms)` forces the database server engine to hold threads open and suspend response processing. By repeatedly sending or lengthening time-delay NoSQL payloads into vulnerable endpoints (such as product review REST APIs), an attacker can cause thread starvation and induce a Denial-of-Service (**DoS**) state on target application servers.

In this exercise, we launch OWASP Juice Shop on Ubuntu (`192.168.0.19:80`), analyze REST network traffic from Windows 11 to discover endpoint **/rest/products/1/reviews**, inject NoSQL sleep payloads (**sleep(2000)**), and verify server-side response delay execution (**sleep(200)**).

Objective

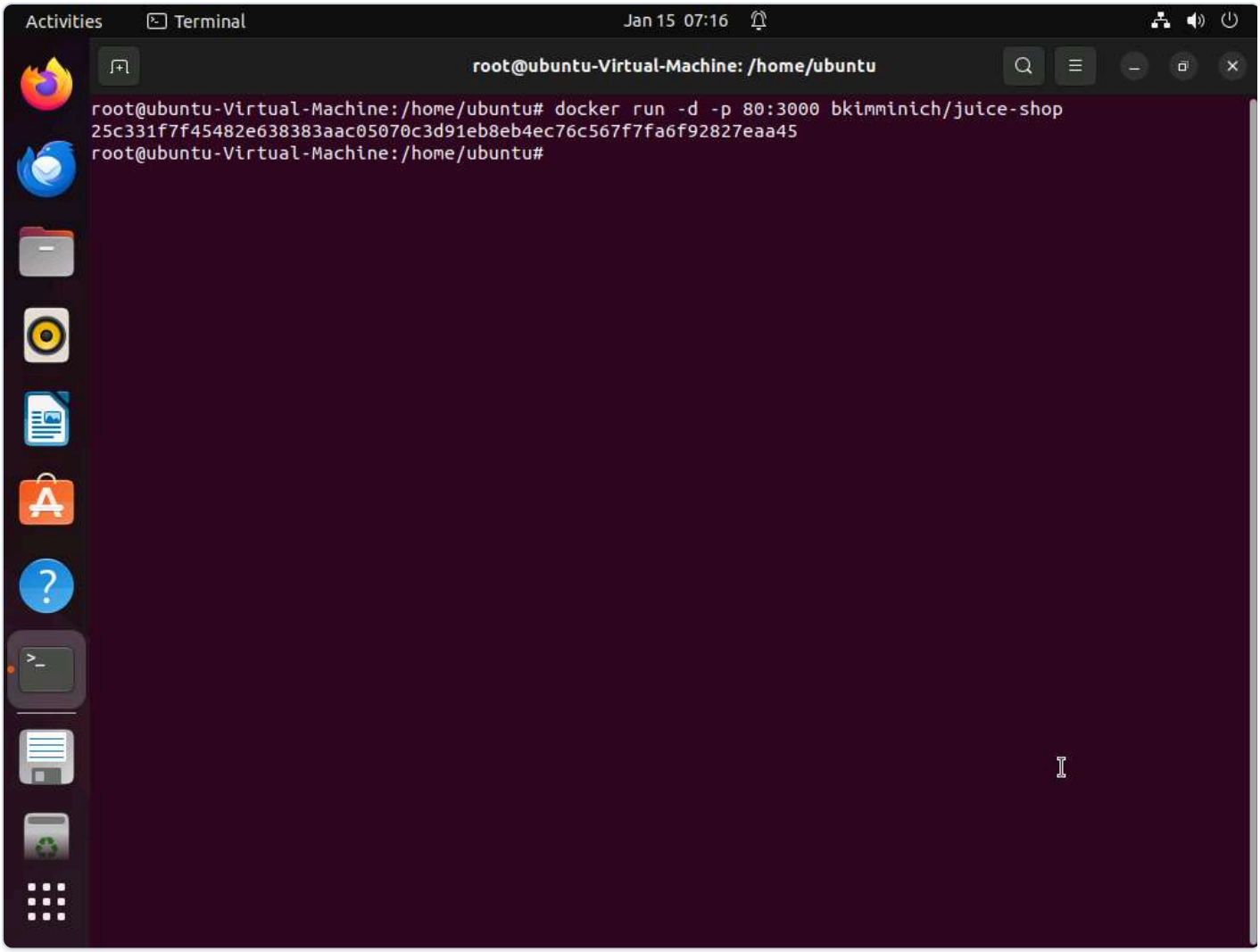
The objective of this lab is to help students learn how to:
- Deploy containerized NoSQL vulnerable applications (OWASP Juice Shop with MongoDB backend)
- Inspect browser developer tools (Network tab) to intercept and enumerate REST API endpoints
- Construct JavaScript-based NoSQL time-delay injection vectors (`sleep(integer_ms)`)
- Execute NoSQL Denial-of-Service (DoS) attacks against REST API parameters (**/rest/products/sleep(2000)/reviews**)
- Calculate payload requirements for specific latency delays (**sleep(200)** for 200 ms)

Task 1: Perform NoSQL Injection DoS on OWASP Juice Shop

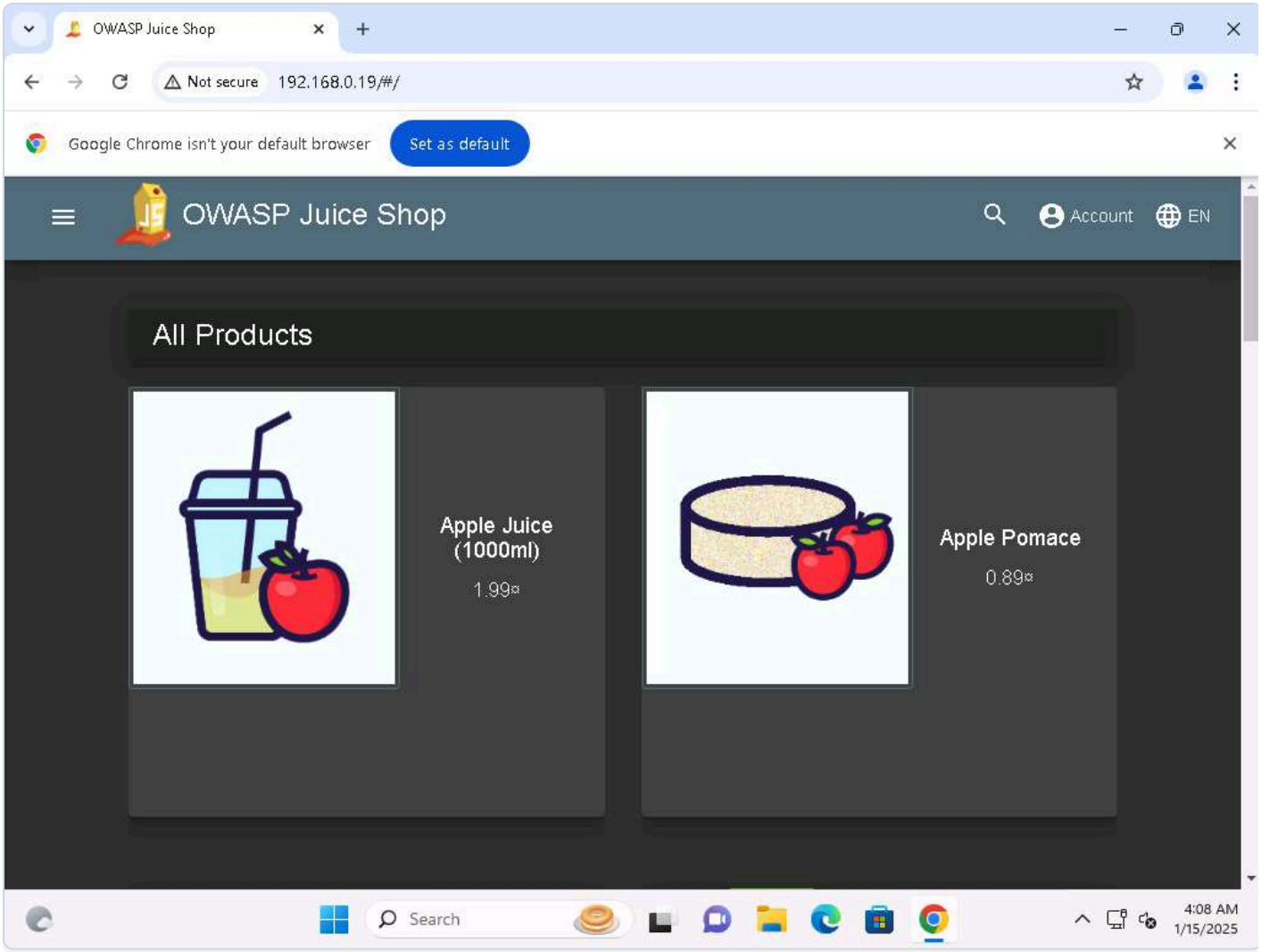
In this task, we initialize the OWASP Juice Shop container on Ubuntu, connect from a Windows 11 workstation, capture API review endpoints using browser Developer Tools, and test NoSQL sleep command injection.

Lab Tasks

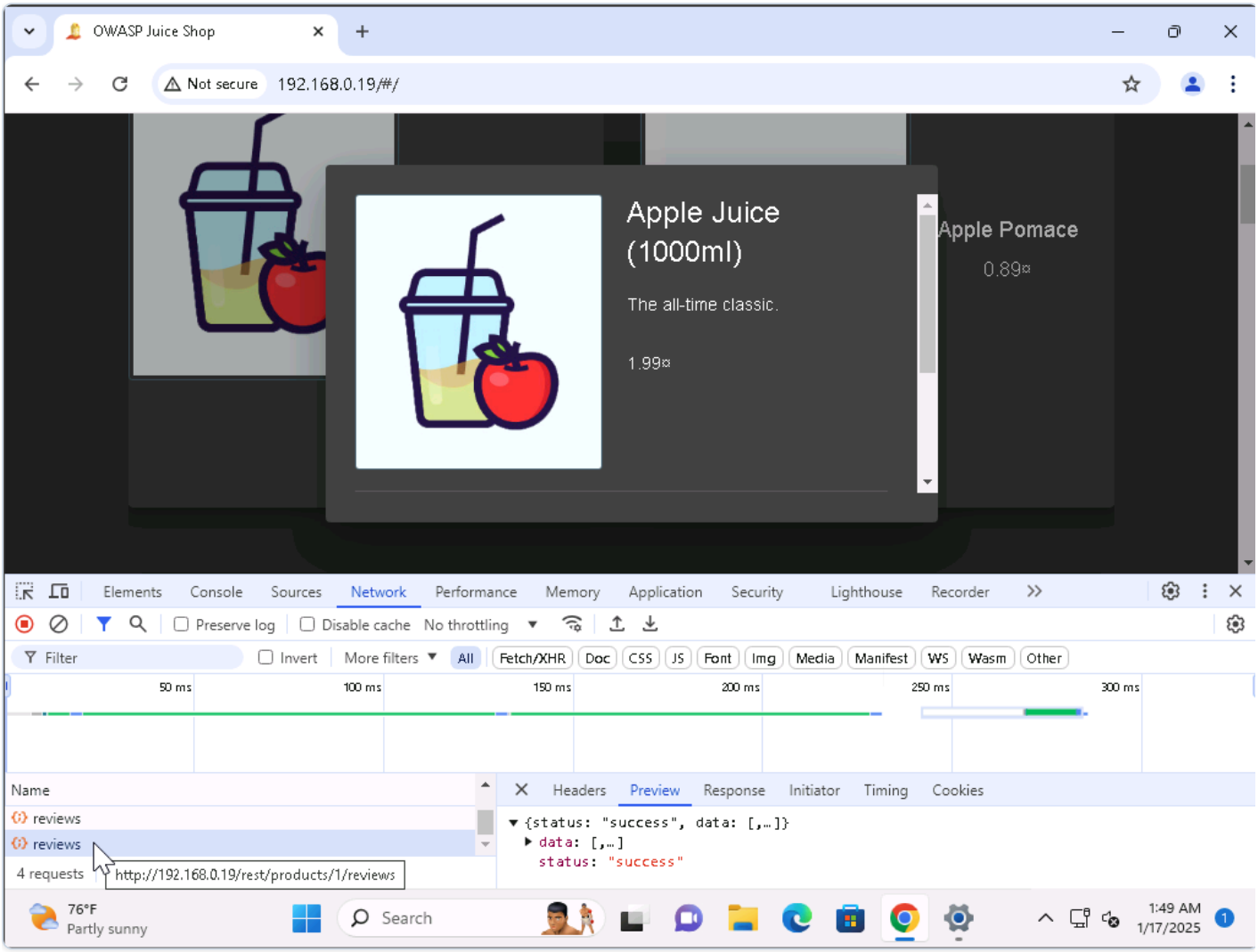
1. Switch to **Ubuntu** (**ubuntu** / **toor**), open a root terminal (**sudo su**), and stop apache2:
   service apache2 stop
2. Launch the OWASP Juice Shop container mapping port 80 to port 3000:
   docker run -d -p 80:3000 bkimminich/juice-shop
3. Start apache2 service if needed:
   service apache2 start
   (Ignore any minor service warnings).



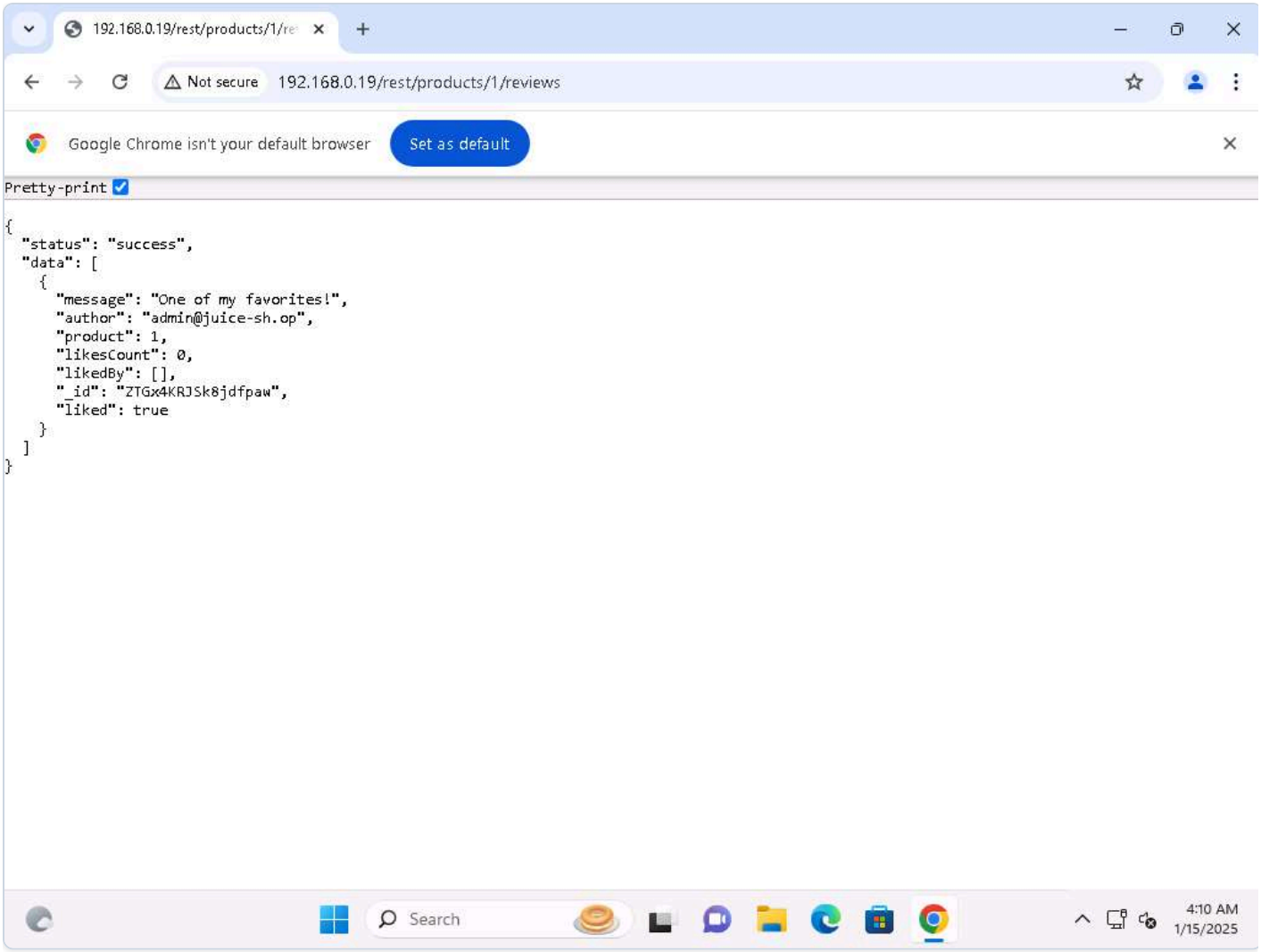
1. Switch to **Windows 11** and log in (**Admin** / **Pa$$w0rd**).
2. Open a web browser and navigate to **http://192.168.0.19:80** to access the OWASP Juice Shop application. Click **Dismiss** on the welcome banner.



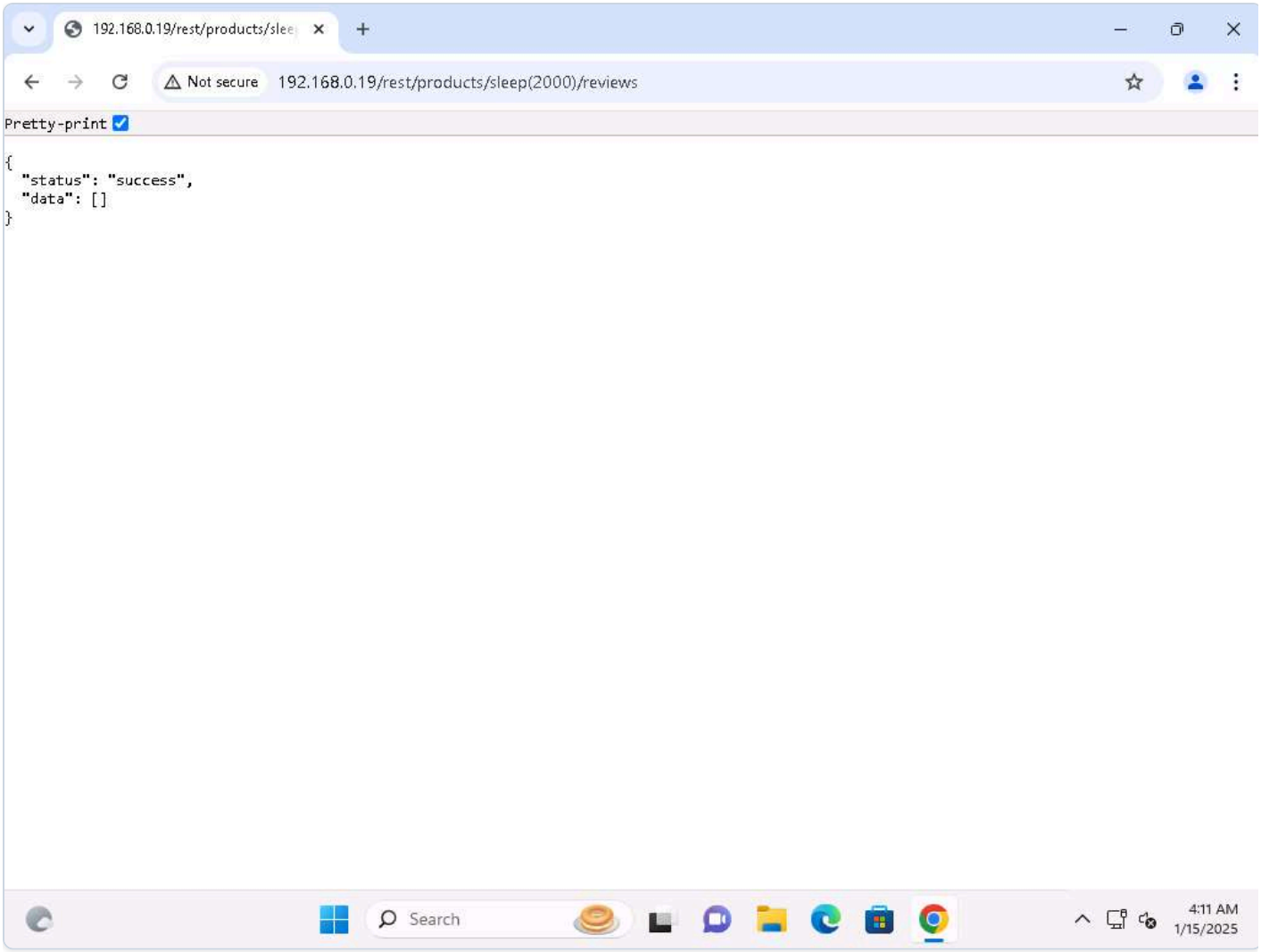
1. Open Browser Developer Tools (**F12**), select the **Network** tab, and reload the page.
2. Click any product item (e.g., *Apple Juice*) and inspect network requests to identify the REST API endpoint:
   /rest/products/1/reviews



1. Verify direct browser access to the product review API endpoint:
   http://192.168.0.19/rest/products/1/reviews



1. Craft and submit a NoSQL injection sleep payload directly into the product ID path parameter:
   http://192.168.0.19/rest/products/sleep(2000)/reviews



1. Observe that the MongoDB backend executes the injected JavaScript function, delaying the HTTP server response by 2000 milliseconds (2 seconds) before returning data.
2. To introduce a **200 ms** delay, the injected payload function is: **sleep(200)**.
3. Close all browser and developer tool windows.

This concludes the lab exercise.

> **IMPORTANT:** ❓ **Question 6.4.1.1** Start the Juice Shop container on an Ubuntu machine and access it from a Windows 11 system. Inject a sleep command into an API query to exploit MongoDB's behavior, causing a delay in execution and consuming server resources. Provide the sleep command required to introduce a 200 ms delay. [!SUCCESS] ✅ **Answer:** **sleep(200)**